

Systèmes d'exploitation I

Chapitre 5 : Gestion de la mémoire

Moufida Maimour

TELECOM Nancy – 2ème année

2025-2026

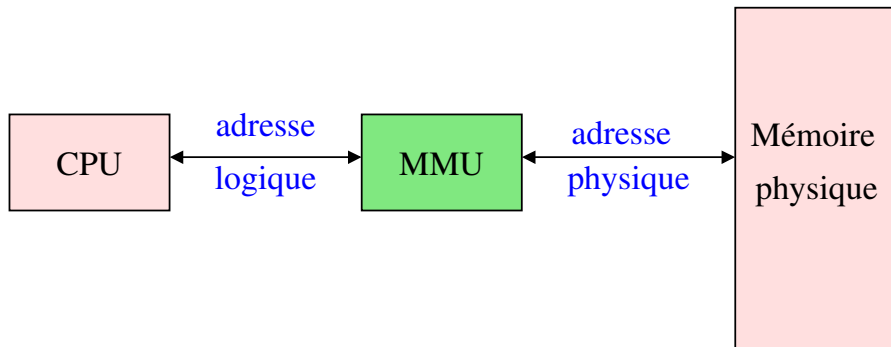
- 1 Notion d'adresse logique
- 2 Pagination et notion de la mémoire virtuelle
- 3 Études de cas : AMD64 (x86), Linux

Objectifs :

- Comprendre les principes fondamentaux de la gestion mémoire (adresses, MMU, hiérarchie)
- Maîtriser les mécanismes modernes : pagination, algorithmes de remplacement, mémoire virtuelle
- Étudier la gestion mémoire dans les OS actuels (x86-64, Linux)

Adresse physique vs. logique

- Adresse physique (réelle) correspond à la vision de la mémoire centrale
→ espace d'adressage physique ou réel
- Adresse logique (virtuelle) est l'adresse générée par l'unité centrale
→ espace d'adressage logique ou virtuel
- Lors de la liaison d'adresses (compilation, chargement ou exécution), les adresses logiques sont traduites en adresses physiques
- la MMU (Memory Management Unit) est un dispositif matériel qui permet la conversion @logique → @physique

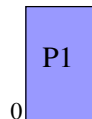


Adresses physiques vs. logiques

Exemple avec 3 processus

Virtual Addresses

View



Physical Addresses

View



Moment de la traduction d'adresses

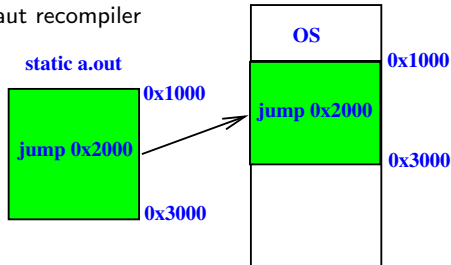
Liaison statique → programme absolu

Traduction lors de la compilation ou l'édition des liens



- Programme **absolu** → **rigide** :

- ▶ ne peut être chargé qu'à une adresse mémoire spécifique
- ▶ si l'adresse de chargement change, il faut recompiler

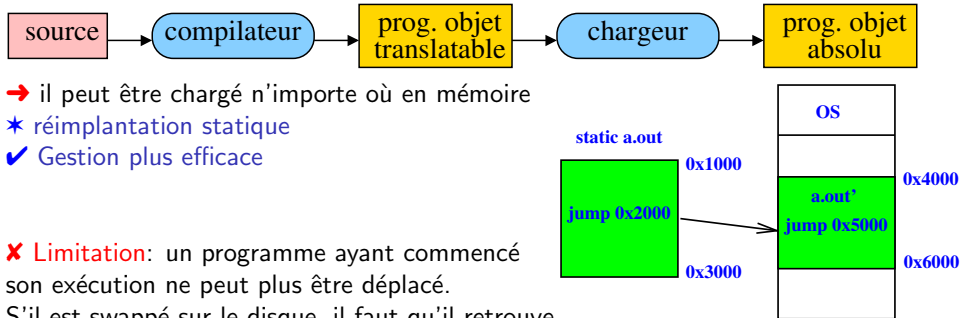


- Utilisation :

- ▶ firmwares ou noyaux OS
- ▶ systèmes embarqués
- ▶ chargement manuel dans des environnements où l'OS ne gère pas la relocalisation dynamique

Moment de la traduction d'adresses

Liaison au chargement → programme translatable



→ il peut être chargé n'importe où en mémoire

★ réimplantation statique

✓ Gestion plus efficace

✗ **Limitation:** un programme ayant commencé son exécution ne peut plus être déplacé.

S'il est swappé sur le disque, il faut qu'il retrouve son emplacement initial : ★ programme non **relogeable**)

★ Rappel.

- `gcc -o prog prog.c → prog` programme objet absolu
- `gcc -c prog.c → prog.o` : programme objet translatable
- `gcc -c -fPIC prog.c → prog.o` : programme objet translatable et relogeable. **ex.** prog. ou lib. qui doit être chargé(e) à différentes @logiques

Moment de la traduction d'adresses

Liaison au moment l'exécution

- Traduction dynamique des adresses durant l'exécution du programme
 - ✓ réimplantation dynamique en cours d'exécution
 - ★ le module résultant est translatable et relogeable
- Implantations possibles (support matériel):
 - ▶ utilisation d'un registre de base → contiguité physique en mémoire
 - ▶ la pagination (sera vue plus tard)

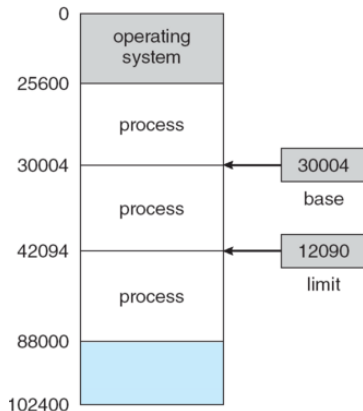
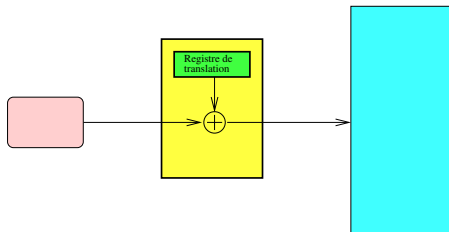
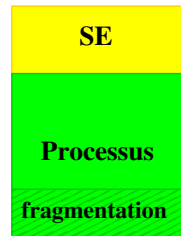


Schéma de gestion de la mémoire

Partition unique

- Un seul processus à la fois en mémoire, encore utilisée lorsqu'un seul programme doit s'exécuter à la fois :
 - ★ certains systèmes embarqués,
 - ★ anciens OS (MS DOS),
 - ★ programme d'installation Linux, ...
- Protection (OS, processus) simple à mettre en œuvre :
 - ★ un registre de base (de translation) et un registre limite



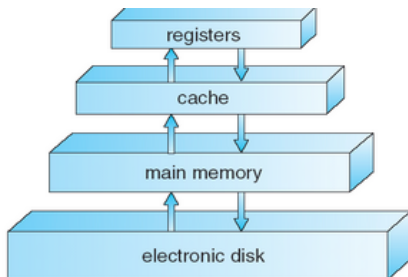
Avantages

- tout le processus est en mémoire
- facile à implanter

Inconvénients

- **fragmentation interne** : espace allouée à un processus mais pas utilisé
- degré de multiprogrammation limité
- mauvaise exploitation de l'UC et la MC

Hierarchie de stockage (1/2)



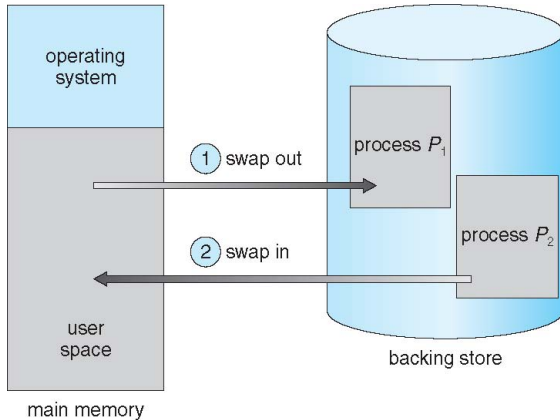
- taille,
- vitesse,
- coût,
- volatilité

Level	Registers	Cache	Main Mem.	Disk	
				SSD	HDD
Typical size	<1Kb	64KB à 96MB	8 à 128GB	up to TB	
Access time	< 1ns	1-15ns	≈ 80ns	100μs	5ms
Bandwidth /s	20-100TB	1T-600GB	200GB	10GB	200MB
Volatile?	Yes	Yes	Yes	No	
Managed by	Compiler	Hardware	OS	OS	
Backed by	Cache	Main Mem	Disk	tape, cloud, ...	

Hiérarchie de stockage (2/2)

- l'UC a un accès direct à ses **registres** et à la **mémoire principale** :
 - ▶ un cycle d'horloge (ou moins) pour accéder à un registre
 - ▶ souvent, plusieurs cycles d'horloge sont nécessaires (suivant le type d'adressage) pour accéder à la mémoire principale
- mémoire plus rapide = mémoire plus petite parce que plus coûteuse
- le **cache** se situe entre les registres et la mémoire principale
- l'accès au disque dur (mémoire secondaire) est plus lent par rapport à la mémoire centrale
- le matériel et le système d'exploitation sont responsables du déplacement des objets le long de cette hiérarchie :
 - ▶ **exemple** : de la mémoire centrale au disque et inversement.

Le swapping



→ La pagination à la demande

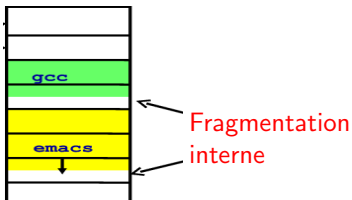
Pagination

✓ Notion de mémoire virtuelle

- Chaque processus croit disposer d'un espace mémoire continu et privé
 - ★ **ex.** dans un système 32 bits, en termes d'adresses logiques, un processus dispose de l'intervalle $[0..2^{32} - 1]$ et une mémoire virtuelle de 4 Go
- L'OS et la MMU traduisent les adresses logiques → physiques

✓ Pagination

- La mémoire physique est découpée en cadres (frames) de taille fixe
- La mémoire virtuelle est découpée en pages de même taille
- Les tables de pages assurent la correspondance (pages ↔ cadres)



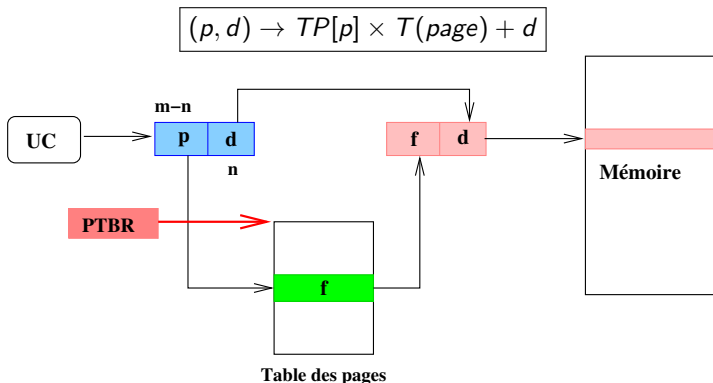
✓ Avantages :

- allocation **non contigüe**
- limite la fragmentation

Pagination

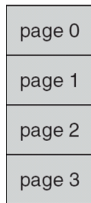
Conversion @logique → @physique

- Machine m bits avec $T(\text{page}) = 2^n$
- La table des pages (TP) est maintenue en mémoire
- Page-table base register (PTBR) pointe vers la table des pages



Pagination

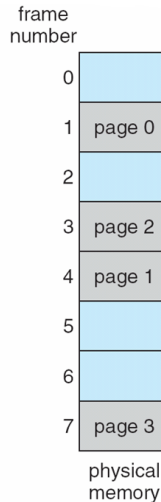
Exemple illustratif



logical
memory

0	1
1	4
2	3
3	7

page table

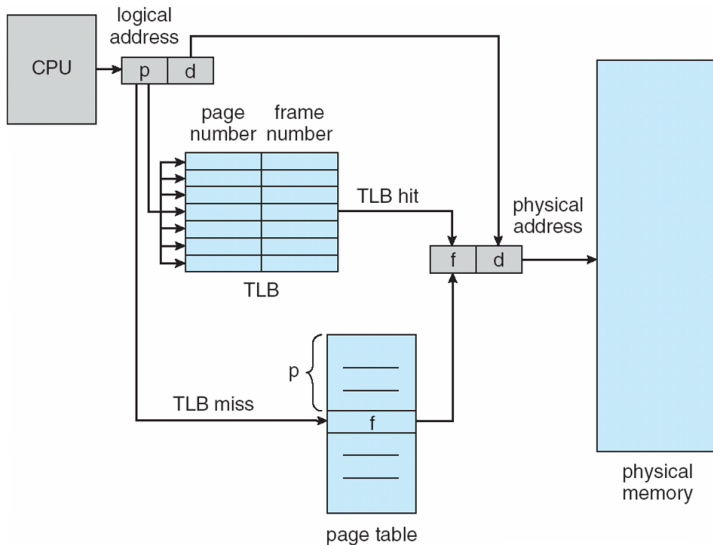


Pagination - TLB (Translation look-aside buffers)

→ conversion plus rapide

- 2 accès mémoire sont nécessaires pour accéder à une donnée.
- **Solution** : utilisation d'une mémoire cache assez rapide appelée **mémoire associative** ou **translation look-aside buffers (TLBs)**
- Certaines TLBs assure la protection inter-processus en mémorisant un identificateur pour chaque processus : **address-space identifiers (ASIDs)**
- $(p, d) \rightarrow (c, d)$:
si p est disponible dans la TLB, alors récupérer c ,
sinon aller à la table des pages en mémoire.

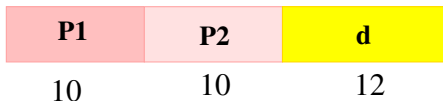
Pagination - TLB (illustration)



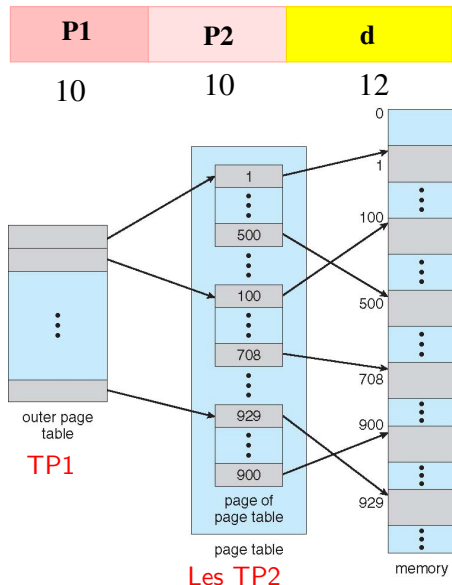
Pagination multiniveaux (1/3)

→ réduire la mémoire (table des pages)

- La table des pages doit être entièrement chargée en mémoire physique.
- **Exemple.** une machine 32 bits avec des pages de 4K.
Une adresse logique : numéro de page (20 bits), déplacement (12 bits)
→ **taille de la TP est $2^{20} \times 4$ octets**
- La pagination multi-niveaux permet de limiter la taille de la table des pages qui doit être résidente en mémoire.
- Aujourd'hui on a jusqu'à 5 niveaux **ex.** x86-64 (plus loin)
- **Exemple simple - cas de 2 niveaux**, une adresse logique prend la forme :



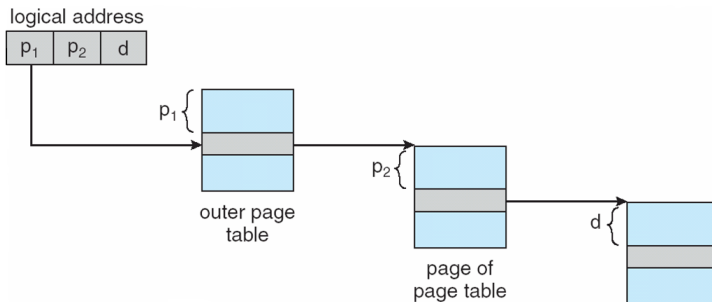
Pagination multiniveaux (2/3)



- ✓ Nombre des entrées de la TP1 : 2^{10}
- ✓ Nombre des entrées d'une TP2 : 2^{10}
- ✓ Théoriquement, un total de :
 $2^{10} + 2^{10} \times 2^{10}$ entrées
- ✓ En pratique, seule la TP1 est nécessaire, le nombre des TP2 dépend des besoins réels d'un processus.
- ★ **ex.** un programme de 2^{10} pages, besoin de seulement $2^{10} + 2^{10} = 2^{11}$ entrées

Pagination multiniveaux (3/3)

Traduction d'adresses



Pagination - Bit (valid/invalid) ou bit de présence

✓ Le bit (valid/invalid) est associé à chaque page pour indiquer si elle est dans l'espace d'adresse réel du processus

Exemple :

★ une machine 14 bits

★ la taille d'une page est $2K = 2^{11}$

→ espace d'adresse ($0 \dots 2^{14} - 1$) de $2^{14}/2^{11} = 8$ pages

→ 8 entrées dans la table des pages

✓ un programme qui n'utilise que les adresses (0-10468)

→ 6 pages

✗ les pages 6 et 7 sont marquées comme étant invalides

frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

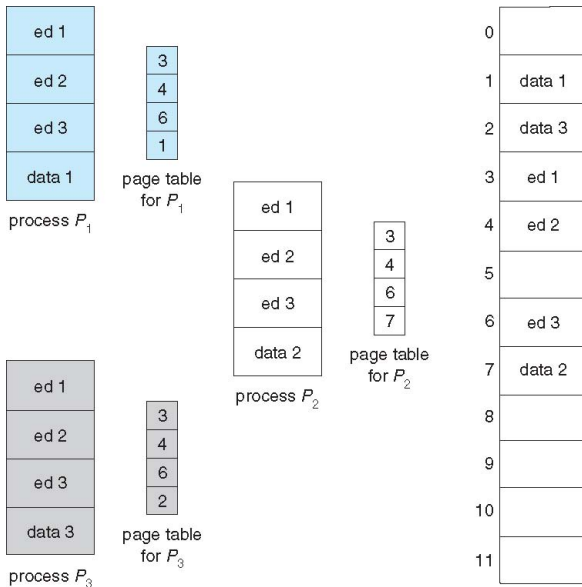
00000

page 0
page 1
page 2
page 3
page 4
page 5

10,468
12,287

0
1
2
3
4
5
6
7
8
9
...
n

Pagination et partage



Pagination à la demande

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

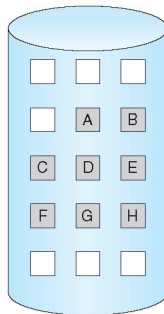
logical memory

valid-invalid bit		
frame		bit
0	4	v
1		i
2	6	v
3		i
4		i
5	9	v
6		i
7		i

page table

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

physical memory



Pagination à la demande

★ Une page est transférée de la mémoire secondaire à la mémoire centrale (MC) lorsqu'on a besoin de cette page.

```
loop
  ins(@,opérandes)
  if ins en mémoire then
    exécuter (ins)
    if ins modifie page then
      TP(page).bit_modif ← 1
    end if
  else
    déroutement(défaut de page)
  end if
end loop
```

défaut_de_page(*page*)

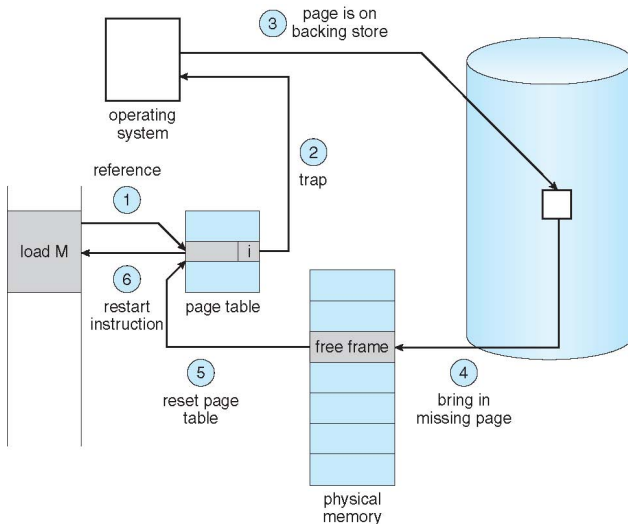
```
if case libre then
  case ← page
else
  choisir (case)
  libérer (case) et y charger (page)
end if
TP(page).case ← case
TP(page).bit_presence ← 1
réexécuter (ins)
```

libérer(*case*) - *case* contient *page_c*

```
TP(pagec).bit_presence ← 0
if TP(pagec).bit_modif = 1 then
  transférer pagec vers la mémoire
  auxiliaire
end if
```

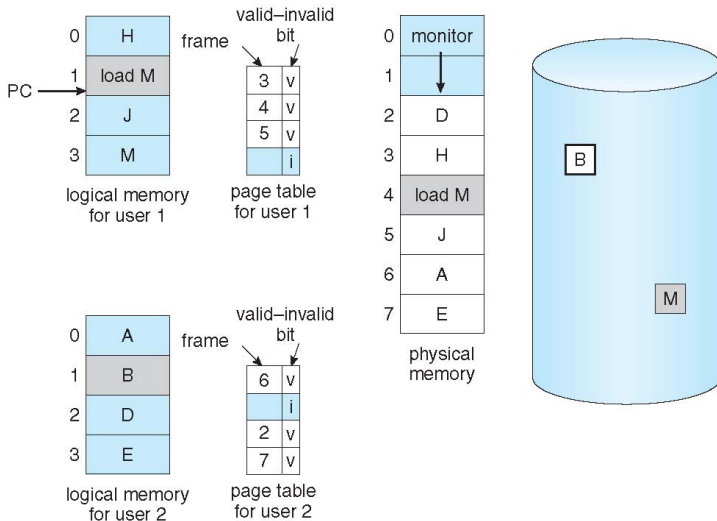
Pagination à la demande

Défaut de page illustré



Algorithmes de remplacement

Besoin de remplacement de pages



Algorithmes de remplacement

L'algorithme optimal

- La page victime est celle qui **mettra** le plus de temps à être référencée.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		2				2						7		
	0	0	0		0		4				0						0		
		1	1		3		3				3						1		

page frames

- Algorithme pratiquement impossible.

Algorithmes de remplacement

Premier arrivé, premier servi : FIFO

- la page victime est celle qui a été chargée la première

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2																
	0	0	0																
			1	1															

page frames

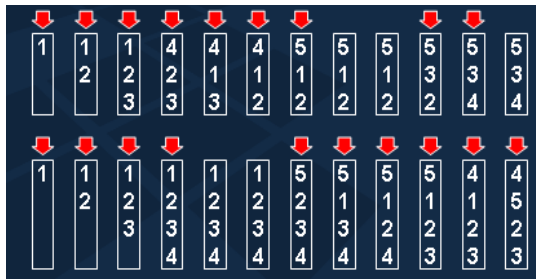
Implémentation

- file d'attente FIFO des pages (la victime en tête de file)

Algorithmes de remplacement

Anomalie de Belady

Soit la chaîne de référence : 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.



- 3 cadres de pages : 9 défauts de pages
- 4 cadres de pages : 10 défauts de pages

★ Avec plus de cadres, on peut avoir plus de défauts de pages !

Algorithmes de remplacement

LRU : Least Recently Used

- la page victime est la moins récemment référencée
- Exemples :**

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

1	1	1	1	5
2	2	2	2	2
3	5	5	4	4
4	4	3	3	3

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1
	0	0	0		0		0	0	3	3		3		0		0
		1	1		3		3	2	2	2		2		2		7

page frames

→ algorithme stable mais nécessite une gestion coûteuse de la liste modifiée à chaque accès à une page

Algorithmes de remplacement

Approximations de LRU

Bit de référence (R)

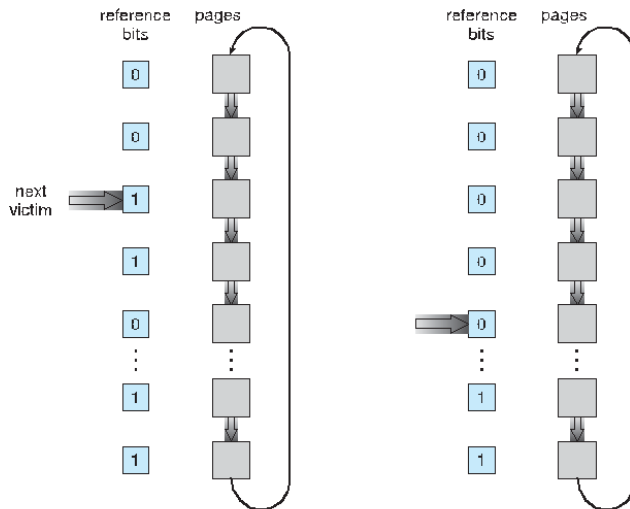
- R est initialement 0 et mis à 1 à chaque référence
- FIFO est appliqué d'abord sur les pages ayant le bit de référence à 0.

La seconde chance

- Le bit de référence est positionné à 1 à chaque référence
- Liste circulaire avec un pointeur sur la prochaine victime
- Si la page à remplacer est telle que $R = 0$, on la remplace sinon on lui donne une 2ème chance :
 - ▶ $R \leftarrow 0$
 - ▶ laisser la page en mémoire
 - ▶ passer à la suivante dans la liste en appliquant les mêmes règles.

Algorithmes de remplacement

La seconde chance



Algorithmes de remplacement

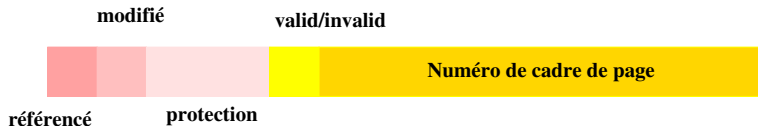
Not Recently Used : NRU

- associer à chaque pages, 2 bits : R et M initialisés à 0
- à chaque accès en lecture : $R \leftarrow 1$
- à chaque accès en écriture : $M \leftarrow 1$
- à chaque interruption d'horloge, l'OS remet R à 0
- Valeurs possibles (MR):
 - ▶ 00 : non modifiée, non référencée
 - ▶ 01 : non modifiée, référencée
 - ▶ 10 : modifiée, non référencée
 - ▶ 11 : modifiée, référencée
- La page victime est choisie au hasard parmi celles ayant le plus petit indice (MR)

Résumé de cette première partie

- Notion d'adresse logique/physique
- Notion de la mémoire virtuelle
- La pagination et la pagination à la demande

Structure possible d'une entrée de la table des page :



- numéro de cadre de page
- bit de **présence** (valid/invalid)
- bit (3 bits) de protection (permissions)
- bit **modifié** (dirty bit): si positionné à 1, la page doit être écrite sur le disque (accédée en écriture par un processus)
- bit **référéncé**: mis à 1 à chaque fois que la page est accédée en lecture ou en écriture → les algorithmes de remplacement de pages

Architecture x86 - 32 bits

Pagination

✓ 32 bits → Mémoire virtuelle : 4GB

✓ **Pagination à 2 niveaux**

→ taille de la page : 4KB

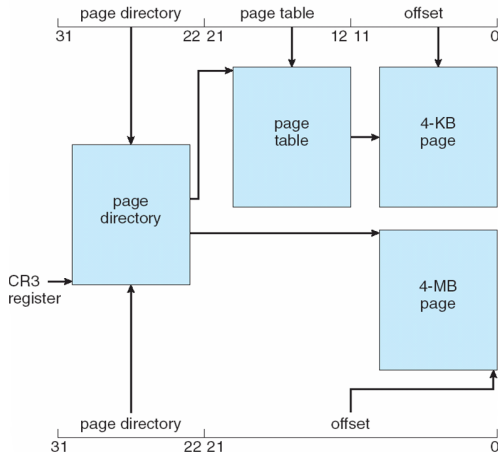
- p.d (10bits), p.t (10bits), offset (12 bits)

✓ **Pagination étendue**

→ taille de la page : 4MB

- activée : flag "Page Size" de l'entrée de la "page directory"
- p.d (10bits), offset (22 bits)

★ cr3 contient l'@physique de la "page directory"



Architecture AMD64 / x86-64

Format des adresses

✓ En théorie : 64 bits d'adresse virtuelle → espace théorique de $2^{64} = 16$ exaoctets.

✗ En pratique : seuls les 48 bits de poids faible (0..47) sont significatifs

→ LA48 :

- les bits (63..48) sont une extension de signe du bit 47 → adresse canonique.
- toute adresse non canonique déclenche une exception (#GP).
- espace utilisateur : [00000000 00000000 - 00007FFF FFFFFFFF] (128 TB).
- espace noyau : [FFFF8000 00000000 - FFFFFFFF FFFFFFFF] (-128 TB).

LA : Linear Address - y a aussi LA57 avec 5 niveaux de pagination

#GA : General Protection fault

Architecture AMD64 / x86-64

Pagination multi-niveaux

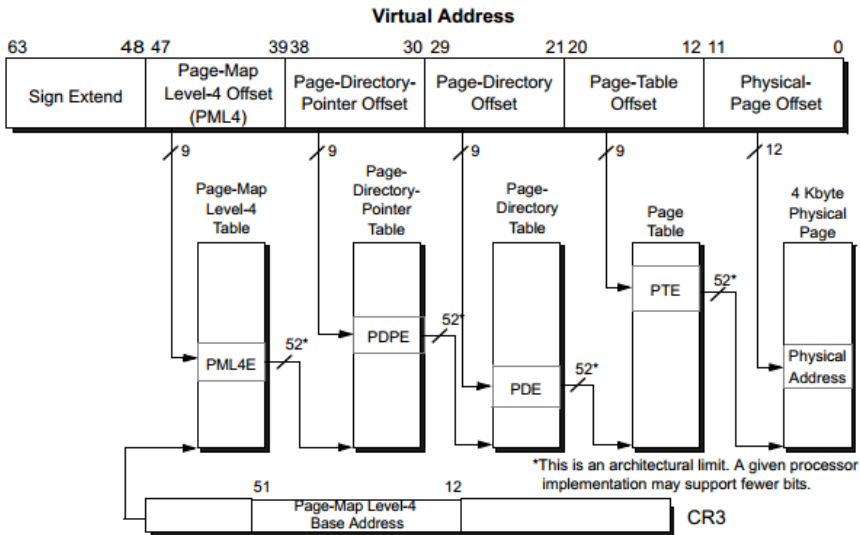
- Taille page standard = 4 KB (4096 octets) et 4 niveaux de pagination :
 - ▶ PML4 (Page Map Level 4) sur 9 bits [47 - 39]
 - ▶ PDPT (Page Directory Pointer Table) sur 9 bits [38 - 30]
 - ▶ Page Directory (PD) sur 9 bits [29 - 21]
 - ▶ Page Table (PT) sur 9 bits [20 - 12]
 - ▶ Offset dans la page 12 bits [11 - 0]
- Extensions :
 - ▶ Pages de 2 MB avec 3 niveaux de pagination.
 - ▶ Pages de 1 GB avec 2 niveaux de pagination.
- Le registre cr3 contient l'adresse physique de la PML4.
- TLB : cache des traductions récentes pour éviter la traversée des 4 tables

Gestion matérielle

- Le registre CR3 contient l'adresse physique de la PML4.
- TLB : cache des traductions récentes pour éviter la traversée des 4 tables.
- Protection : chaque entrée de la PT
 - ▶ P (Present) : page présente en mémoire ?
 - ▶ R/W : protection en écriture.
 - ▶ U/S (User/Supervisor) : réservé au noyau ?
 - ▶ NX (No eXecute) : interdit l'exécution de code dans la page

Cas de l'architecture x86-64

Traduction d'adresse

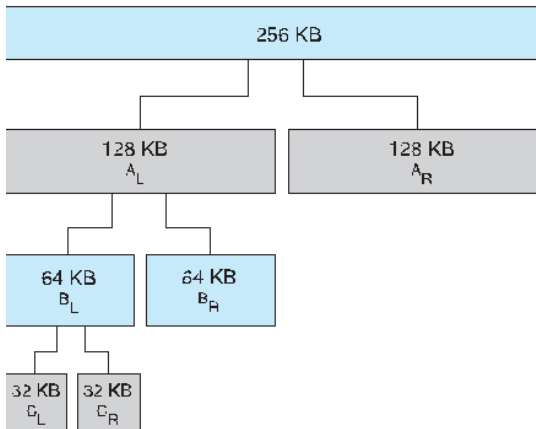


Source AMD Architecture Programmer's Guide, Vol. 2, Rev 3.23, 2013, page 132

Cas de Linux

Algorithme des frères siamois (Buddy Allocator)

✓ Buddy allocator : combine/sépare des blocs de 2^n pages pour limiter la fragmentation. [Donald KNUTH - 1973]



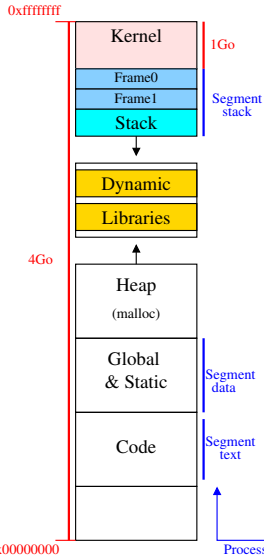
Cas Linux - Les régions (VMAs)

- ✓ Une **région** (Virtual Memory Area - VMA) contient un ensemble de pages consécutives possédant les mêmes propriétés de protection : région de code, région de pile ...
- ✓ Chaque processus possède une table de VMAs - liste de descripteurs de régions

Structure (partielle) d'un descripteur de région (`vm\area\struct`) :

- **vm_start, vm_end**: adresse de début et fin de la région.
- **vm_next**: pour relier toutes les régions d'un processus
- **vm_flags** : un ensemble de flags décrivant les propriétés et la protection de la région :
 - ▶ VM_READ
 - ▶ VM_WRITE
 - ▶ VM_EXEC
 - ▶ VM_SHARED
 - ▶ ...

Cas Linux 32 bits - Mémoire virtuelle d'un processus



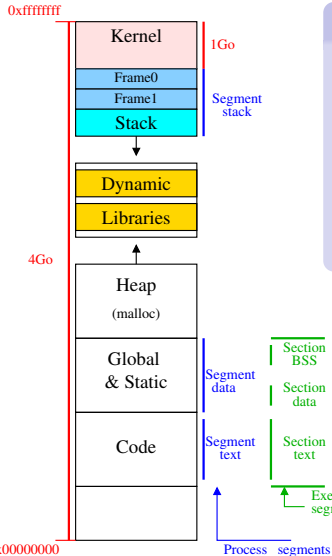
```
cat /proc/self/maps
```

```
08048000-0804c000 r-xp ... /bin/cat
0804c000-0804d000 rw-p ... /bin/cat
0804d000-0806e000 rwxp ...
40000000-40016000 r-xp ... /lib/ld-2.3.2.so
40016000-40017000 rw-p ... /lib/ld-2.3.2.so
40017000-40018000 rw-p ...
40027000-4014f000 r-xp ... /lib/libc-2.3.2.so
4014f000-40157000 rw-p ... /lib/libc-2.3.2.so
40157000-4015a000 rw-p ...
bffffe000-c0000000 rwxp ...
```

- **.text segment :**

- ▶ de taille fixe
- ▶ contient le code → exécutable
- ▶ accessible en lecture seule →
"segmentation fault" si accès en écriture !

Cas Linux 32 bits - Mémoire virtuelle d'un processus



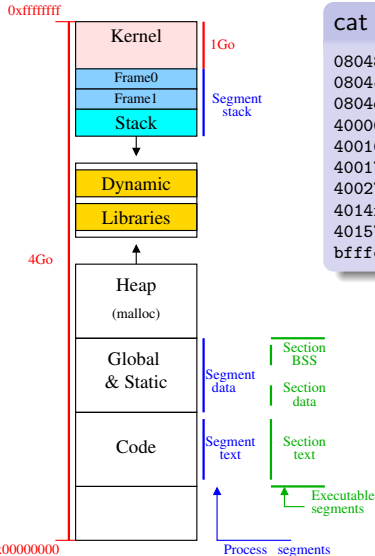
cat /proc/self/maps

```
08048000-0804c000 r-xp ... /bin/cat
0804c000-0804d000 rw-p ... /bin/cat
0804d000-0806e000 rwxp ...
40000000-40016000 r-xp ... /lib/ld-2.3.2.so
40016000-40017000 rw-p ... /lib/ld-2.3.2.so
40017000-40018000 rw-p ...
40027000-4014f000 r-xp ... /lib/libc-2.3.2.so
4014f000-40157000 rw-p ... /lib/libc-2.3.2.so
40157000-4015a000 rw-p ...
bffffe000-c0000000 rwxp ...
```

- Allocation des données :

- ▶ **statique** pour les variables globales et statiques → data segment :
 - ★ .data, si initialisées
 - ★ .bss, sinon
- ▶ **dynamique** → heap
- ▶ **automatique** → stack segment

Cas Linux 32 bits - Mémoire virtuelle d'un processus



cat /proc/self/maps

```
08048000-0804c000 r-xp ... /bin/cat
0804c000-0804d000 rw-p ... /bin/cat
0804d000-0806e000 rwxp ...
40000000-40016000 r-xp ... /lib/ld-2.3.2.so
40016000-40017000 rw-p ... /lib/ld-2.3.2.so
40017000-40018000 rw-p ...
40027000-4014f000 r-xp ... /lib/libc-2.3.2.so
4014f000-40157000 rw-p ... /lib/libc-2.3.2.so
40157000-4015a000 rw-p ...
bffffe000-c0000000 rwxp ...
```

- ★ le tas et la pile sont de taille var. → évoluent dans 2 sens opposés
- ★ le tas → les données dynamiques (malloc)
- ★ la pile → réalisation des fonctions, stocke principalement les **variables locales** de chaque fonction dans une *frame*.
- ★ l'exécutable → les sections *.text*, *.data* et *.bss*

Cas Linux - Pagination à la demande

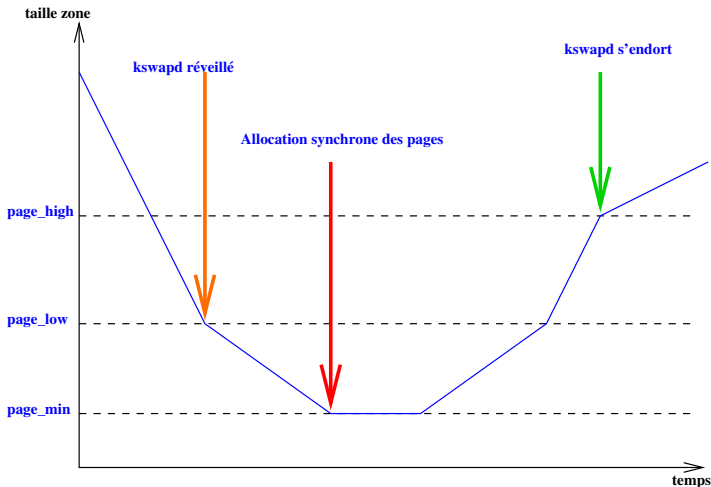
- ✓ les pages sont chargées uniquement quand elles sont réellement utilisées.
- ✓ un espace **swap** (fichier ou partition) peut être utilisé pour libérer de la RAM
- ✗ la page n'est pas présente en mémoire → exception (processeur) **page fault**
- ✗ page dont les bits de permission ne permettent pas l'accès → exception (processeur) **segmentation fault** - signal (OS) **SIGSEGV**

✓ Algorithme de remplacement de pages :

- LRU classique simple mais a des limites sur les gros workloads (bases de données, conteneurs, cloud)
- LRU-GEN / Multi-Gen LRU :
 - ▶ chaque page est associée à une génération qui reflète sa récence d'utilisation
 - ▶ plusieurs listes (multi-gen) plutôt que seulement 2.
 - ▶ une page est promue dans une génération plus récente à chaque accès
 - ▶ s'il faut libérer de la mémoire, l'OS choisit les pages dans les générations les plus anciennes.
- ✓ décision plus fine avec moins d'évictions inutiles

Cas Linux - kswapd & les flushers

→ Garder un minimum de pages libres.



✓ **Writeback** : écriture des "dirty" pages assurée par des **threads flusher** (typiquement un par périphérique)

Cas Linux - Fichiers mappés et mémoire partagée

→ Fichiers mappés (memory-mapped files)

- syscall `mmap()` crée une nouvelle VMA associée à un fichier
- les pages correspondantes sont chargées à la demande (demand paging).
- l'accès au fichier revient à un accès mémoire direct : pas besoin de faire des `read()/write()`, le noyau gère la synchronisation via le **page cache** :
 - ▶ contient les blocs de fichiers récemment utilisés
 - ▶ évite les accès disques fréquents : accélère fortement les I/O
 - ▶ permet le partage entre plusieurs processus
- Les pages modifiées sont marquées "dirty" et écrites plus tard sur disque (flush).

→ Mémoire partagée

- mécanisme IPC permettant à plusieurs processus d'accéder à une même zone mémoire
- syscall `shm_open()` permet la création d'un objet mémoire partagé

Cas Linux

Copy-On-Write (COW)

Implémentation du fork()

- Lors d'un fork(), le fils hérite des mêmes régions mémoire que le père
- Les pages physiques sont marquées en lecture seule
- Si l'un des deux écrit dans une page :
 - page fault
 - le noyau duplique la page pour ce processus.
- **Résultat** : allocation différée, économie de mémoire et meilleure performance